

Dokumentacja Projektu

Jakub Kuczkowski

3 czerwca 2026

Spis treści

1	Wstęp	2
2	Obsługa programu	5
2.1	Metodyka wyzwalania pomiarów i kontroli czasu eksperymentu	5
2.2	Uruchamianie programu	6
2.2.1	Uruchamianie części Arduino	6
2.2.2	Uruchamianie części systemowej	6
2.3	Opis interfejsu użytkownika	6
2.4	Opis błędów oraz rozwiązania	10
3	Zaimplementowana logika systemu	13
3.1	Logika agregacji danych	13
3.2	Logika filtrowania i odrzucania wartości anomalnych	15
4	Opis sprzętu oraz połączeń	17
5	Alternatywne Zastosowania Systemu Pomiarowego	19
5.1	Badanie i Porównywanie Poziomych (Pionowych) Oscylatorów Harmonicznych	19
6	Podsumowanie	21
	Informacje o repozytorium	21

1 Wstęp

Projekt został przygotowany w celu stworzenia uniwersalnego, cyfrowego systemu do precyzyjnego mierzenia i analizowania ruchu ciał w czasie rzeczywistym. Głównym założeniem było zastąpienie niedokładnych, ręcznych metod pomiarowych (takich jak tradycyjne stopery) nowoczesnym układem automatycznym, który eliminuje błędy ludzkiego refleksu i dostarcza precyzyjne dane laboratoryjne.

Zastosowanie projektu obejmuje przede wszystkim szkolne i akademickie laboratoria fizyczne. System pozwala na dokładne badanie kinematyki i dynamiki różnych zjawisk – od analizy ruchu jednostajnie przyspieszonego wózków na równi pochyłej, aż po badanie okresu i amplitudy drgań poziomych oscylatorów harmoniczych (ciężarków na sprężynach). Dzięki funkcji nakładania i ręcznego przesuwania wykresów, oprogramowanie umożliwia uczniom bezpośrednie i czytelne porównywanie parametrów ruchu dla różnych konfiguracji sprzętowych (np. zmiana kąta nachylenia toru, zmiana masy ciała lub sztywności sprężyny).

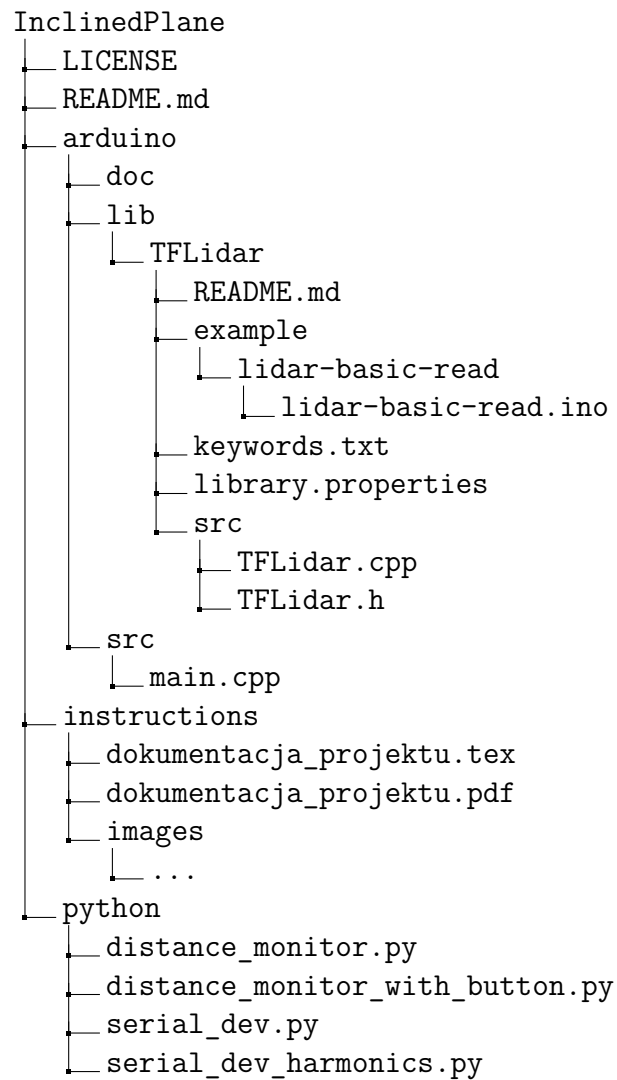
Środowisko pracy systemu składa się ze stanowiska sprzętowego oraz komputera PC. Część sprzętowa to miniaturowy czujnik laserowy LIDAR (działający w technologii Time-of-Flight) oraz fizyczny przycisk wyzwalający, które są połączone z mikrokontrolerem Arduino. Część programowa to aplikacja analityczna z graficznym interfejsem użytkownika, uruchamiana na komputerze w systemie operacyjnym Linux lub Windows. Komunikacja między komputerem a Arduino odbywa się za pomocą standardowego kabla USB przez port szeregowy (UART) z prędkością transmisji 115200 `bodów`.

Struktura projektu

Repozytorium projektu zostało zorganizowane w logiczną strukturę modułową, która precyzyjnie rozdziela warstwę sprzętową (kod mikrokontrolera), oprogramowanie komputerowe (aplikację monitorującą w Pythonie) oraz kompletną dokumentację. Poniżej przedstawiono szczegółowy opis roli i zawartości poszczególnych plików wchodzących w skład systemu.

- **LICENSE** – Plik z licencją określającą zasady korzystania z kodu.
- **README.md** – Instrukcja startowa w formacie tekstowym. Opisuje, jak szybko włączyć program i czego potrzeba do jego uruchomienia.
- **Katalog arduino/** – Zawiera kod dla płytki Arduino, która steruje laserem i przyciskiem.
 - **doc/** – Folder na notatki, schematy kabli i opisy techniczne sprzętu.
 - **lib/TFLidar/** – Gotowa biblioteka (sterownik) do obsługi czujnika laserowego. Zawiera pliki kodu (`TFLidar.cpp`, `TFLidar.h`) oraz przykład (`lidar-basic-read.ino`), który pozwala sprawdzić, czy sam laser działa.
 - **src/main.cpp** – Główny program dla mikrokontrolera Arduino. Odpowiada za bezblokowy odczyt odległości z czujnika LIDAR na pełnej prędkości oraz sprzętową filtrację styków fizycznego przycisku (*debouncing*) na bazie czasu systemowego, bez użycia wstrzymujących procesor funkcji opóźniających.

UWAGA: Zarówno ta biblioteka, jak i plik `main.cpp` muszą zostać wgrane na płytkę Arduino za każdym razem, gdy zmienisz ich zawartość lub gdy podłączysz zupełnie nową płytkę. Więcej o tym, jak to zrobić krok po kroku, poszukaj na oficjalnej stronie internetowej Arduino w sekcji dokumentacji lub w podstawowych poradnikach dla początkujących użytkowników Arduino IDE.
- **Katalog instructions/** – Folder z dokumentami i instrukcją dla użytkownika.
 - `dokumentacja_projektu.tex` – Plik źródłowy tego dokumentu w systemie $\text{\LaTeX}$.
 - `dokumentacja_projektu.pdf` – Gotowa do wydruku instrukcja w formacie PDF.
 - **images/** – Folder na zdjęcia i zrzuty ekranu z programu, które widać w instrukcji.
- **Katalog python/** – Zawiera programy na komputer, które odbierają dane, uśredniają je i rysują wykres.
 - `distance_monitor.py` – Podstawowa wersja programu monitorującego z graficznym interfejsem użytkownika (GUI).
 - `distance_monitor_with_button.py` – Główna i pełna wersja programu.
 - `serial_dev.py` – Program testowy. Uduje podłączone Arduino i wysyła sztuczne dane, co pozwala testować wygląd wykresu na komputerze bez podłączania prawdziwego sprzętu.



Rysunek 1: Struktura projektu.

Repozytorium **GitHub**: <https://github.com/Z1moRG/InclinedPlane>

2 Obsługa programu

2.1 Metodyka wyzwalania pomiarów i kontroli czasu eksperymentu

Podstawą poprawnego przeprowadzenia każdego doświadczenia z zakresu kinematyki lub dynamiki jest precyzyjna kontrola momentu rozpoczęcia oraz zakończenia rejestracji danych. W klasycznych laboratoriach fizycznych zjawisko asynchroniczności (opóźnienia między ręcznym włączeniem stopera a fizycznym puszczeniem badanego ciała) jest głównym źródłem błędów pomiarowych i utrudnia późniejszą analizę porównawczą wykresów. Aby wyeliminować ten problem, omawiany system pomiarowy został zaprojektowany w sposób elastyczny i pozwala na wyzwalanie rejestracji danych na dwa całkowicie niezależne sposoby: programowy oraz sprzętowy.

Pierwszy sposób to wyzwalanie czysto programowe, realizowane bezpośrednio z poziomu komputera zarządzającego eksperymentem. Jest ono dedykowane do doświadczeń, w których badacz ma pełną swobodę czasową i nie musi fizycznie asystować przy samym torze pomiarowym w ułamku sekundy, w którym rusza rejestracja. Metoda ta doskonale sprawdza się w badaniach obiektów poruszających się cyklicznie, takich jak oscylatory harmoniczne, gdzie ruch trwa nieprzerwanie, a moment włączenia zapisu nie wpływa na ostateczny kształt rejestrowanej amplitudy czy okresu drgań.

Drugi sposób to wyzwalanie sprzętowe, które wykorzystuje fizyczny przycisk zaimplementowany bezpośrednio na płycie mikrokontrolera stanowiska badawczego. Podejście to całkowicie zmienia model interakcji z urządzeniem i zamienia przycisk w sprzętowy pilot zdalnego sterowania o charakterze automatycznego wyzwalacza pomiaru. Idea ta opiera się na eliminacji błędów ludzkiego refleksu w doświadczeniach ściśle dynamicznych, takich jak zjazd wózka z równi pochyłej. Użytkownik, zamiast operować myszką komputera, ręcznie blokuje wózek na szczycie toru, jednocześnie wciskając fizyczny przycisk. Wciśnięcie przycisku informuje program, że układ znajduje się w stanie uzbrojenia i oczekiwania. W ułamku sekundy, w którym użytkownik puszcza wózek i zwalnia blokadę, przycisk wysyła do komputera sygnał o zmianie stanu elektrycznego. System natychmiast i samoczynnie inicjalizuje zbieranie danych z lasera. Ponieważ moment ruszenia ciała i moment startu pomiaru są sprzętowo zsynchronizowane, wykres generuje się bez żadnych martwych faz oczekiwania, co ma fundamentalne znaczenie dla dokładności naukowej eksperymentu.

Równie istotnym filarem logiki systemu jest koncepcja dynamicznego *timeoutu*, czyli ręcznie definiowanego czasu trwania pomiaru. W fizyce laboratoryjnej czas trwania obserwacji musi być ściśle dopasowany do dynamiki badanego zjawiska. Za długa rejestracja skutkuje zbieraniem szumu informacyjnego po zatrzymaniu się obiektu, co zniekształca skalę wykresu, z kolei czas zbyt krótki może uciąć kluczową fazę ruchu. Użytkownik przed każdym przejazdem ma możliwość elastycznego określenia długości trwania okna pomiarowego w milisekundach. Pozwala to badaczowi na strategiczne zaplanowanie doświadczenia - od ultrakrótkich, dynamicznych sesji trwających zaledwie dwie sekundy dla stromych równi pochyłych, aż po długie, kilkudziesięciosekundowe obserwacje powolnego zanikania drgań w tłumionych oscylatorach harmonicznych. Po upływie tego precyzyjnie zdefiniowanego czasu, niezależny zegar systemowy komputera bezwarunkowo przerywa pętlę pomiarową, automatycznie zamyka serię i przywraca stanowisko do stanu pełnej gotowości na kolejny etap badań.

2.2 Uruchamianie programu

2.2.1 Uruchamianie części Arduino

Aby przygotować część sprzętową do pracy, należy wykonać następujące kroki:

1. Podłączyć Arduino do komputera za pomocą standardowego kabla USB. Komputer powinien automatycznie wykryć urządzenie i przypisać mu odpowiedni port komunikacyjny (COM na systemie Windows lub `ttyACM/ttyUSB` na systemie Linux).
2. Sprawdzić poprawność podłączenia wszystkich pinów. Czujnik laserowy LIDAR musi mieć prawidłowo wpięte cztery przewody: zasilanie do pinu 5V, masę do pinu GND, oraz kable sygnałowe (zielony i biały) odpowiednio do pinów 10 i 11 mikrokontrolera. Dodatkowo fizyczny przycisk startowy musi być połączony z pinem 4 oraz masą (GND). Złe połączenie lub luźne kable spowodują, że program nie odbierze danych z lasera lub nie zareaguje na puszczenie wózka.
3. Upewnić się, czy oprogramowanie jest wgrane na płytkę. Jeśli jest to nowe Arduino, zawartość plików została zmieniona lub czujnik nie wysyła danych, należy otworzyć program Arduino IDE i ponownie wgrać program na płytkę.

2.2.2 Uruchamianie części systemowej

Aplikację na komputerze można uruchomić na dwa sposoby, w zależności od przygotowania stanowiska:

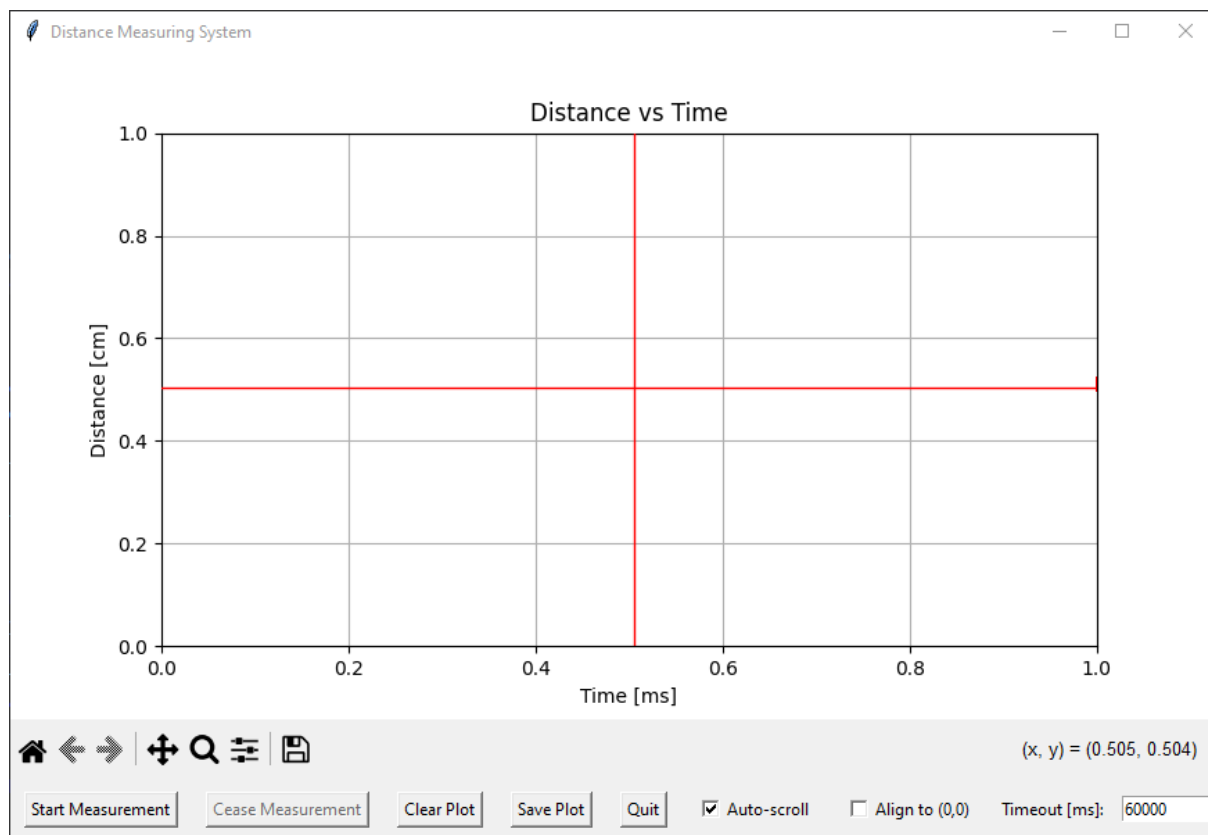
- Uruchomienie bezpośrednio przez plik Python. Wymaga to zainstalowanego środowiska Python 3 na komputerze. Przed włączeniem programu należy upewnić się, że pobrano wszystkie wymagane biblioteki (takie jak `PySerial` do komunikacji oraz `Matplotlib` do rysowania wykresów) i środowisko jest poprawnie skonfigurowane. Program włącza się, uruchamiając główny plik `distance_monitor_with_button.py`.
- Uruchomienie przez gotowy plik wykonywalny EXE. Jest to najszybsza metoda, która nie wymaga instalowania Pythona ani żadnych dodatkowych bibliotek na komputerze. Wystarczy wejść do katalogu programu i kliknąć dwukrotnie myszką na plik wykonywalny z rozszerzeniem `.exe`. Program włączy się od razu w gotowym oknie graficznym, samoczynnie przeszuka porty USB i połączy się z działającym Arduino.

2.3 Opis interfejsu użytkownika

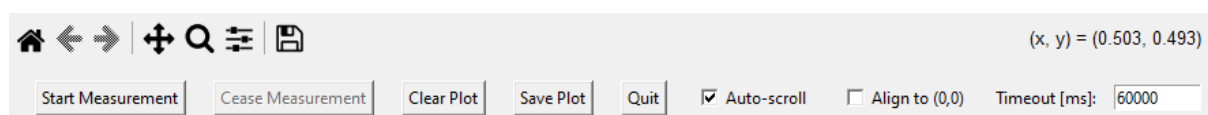
Program stanowi intuicyjne, graficzne centrum laboratoryjne służące do rejestracji i analizy zjawisk kinematycznych w czasie rzeczywistym. Cała interakcja z systemem pomiarowym odbywa się za pomocą jednego, zintegrowanego okna graficznego, co eliminuje konieczność znajomości środowisk programistycznych czy obsługi konsoli tekstowej.

Po uruchomieniu aplikacji system automatycznie przeprowadza skanowanie portów USB komputera w poszukiwaniu podłączonej płytki Arduino z czujnikiem LIDAR. Jeśli urządzenie zostanie wykryte, użytkownik od razu widzi przygotowany, czysty układ współrzędnych (Droga w funkcji Czasu) z widoczną siatką pomocniczą, etykietami osi oraz dynamicznym celownikiem pomiarowym, który w postaci czerwonego krzyża precyzyjnie

podąża za kursorem myszy, ułatwiając natychmiastowy, wzrokowy odczyt parametrów z wykresu.

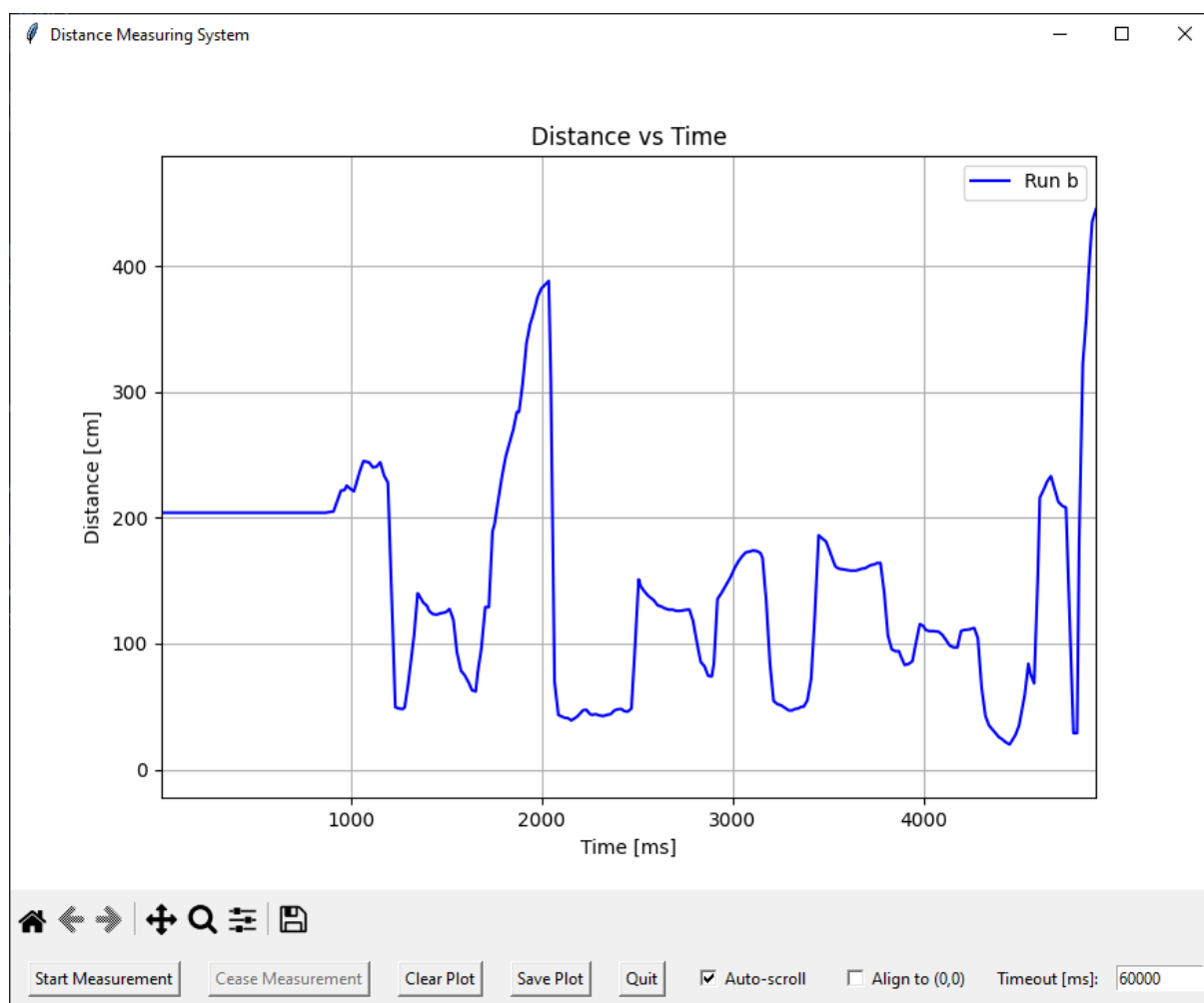


Rysunek 2: Wygląd programu tuż po uruchomieniu.



Rysunek 3: Pasek narzędzi; współrzędne (x, y) widoczne w prawym górnym rogu paska, to współrzędne punktu nad którym znajduje się celownik.

W celu rozpoczęcia eksperymentu (np. puszczenia wózka na równi pochyłej) użytkownik klika przycisk **Start Measurement**. W tym momencie aplikacja automatycznie blokuje możliwość przypadkowego ponownego uruchomienia pomiaru oraz zabezpiecza strukturę wykresu przed usunięciem. Wykres ożywa - napływające z lasera dane są filtrowane i rysowane w czasie rzeczywistym. Dzięki domyślnie aktywnej funkcji **Auto-scroll** kamera wykresu samodzielnie przesuwa się i dopasowuje granice osi tak, aby czoło rysowanej linii (najnowszy punkt) oraz cała trajektoria ruchu były idealnie widoczne na ekranie z zachowaniem czytelnych marginesów.



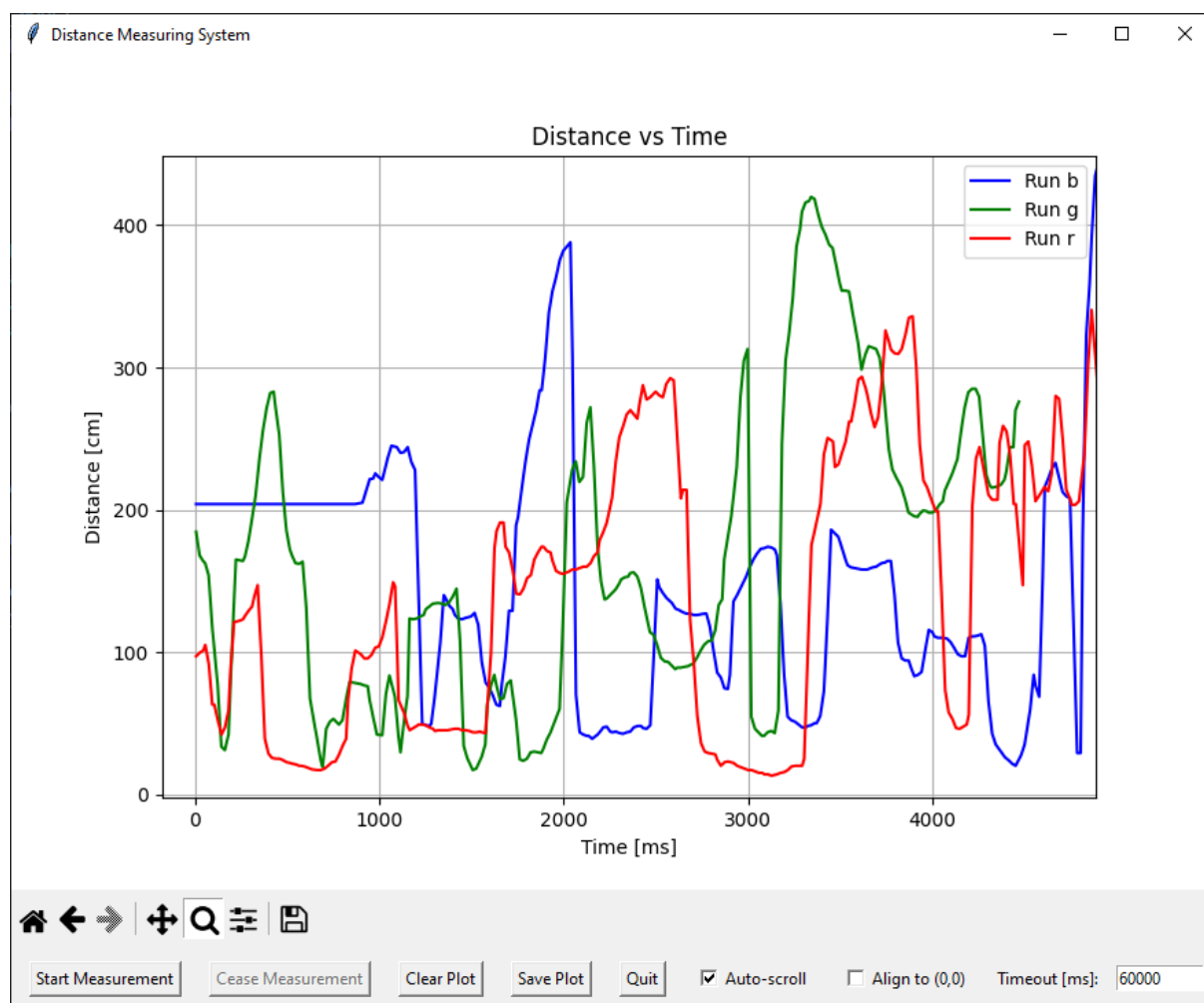
Rysunek 4: W trakcie niektórych procesów, część przycisków jest zablokowana. Tutaj: w trakcie odczytu pomiarów nie można ponownie rozpocząć pomiaru ani wyczyścić wykresu bez wcześniejszego zatrzymania procesu.

Użytkownik ma pełną swobodę w kontrolowaniu sposobu wyświetlania danych. W dowolnym momencie może odznaczyć opcję **Auto-scroll**, co powoduje textitzamrożenie pozycji kamery. Pozwala to na użycie rolki myszy do intuicyjnego przybliżania i oddalania dowolnego fragmentu wykresu (zoom) względem pozycji kursora lub skorzystanie z systemowego paska narzędzi (ruchoma ręczka do przesuwania widoku, lupa). Co ważne, operacje te nie przerywają zbierania danych w tle – nowo zarejestrowane punkty są stale nanoszone na wykres, a ponowne zaznaczenie checkboxa **Auto-scroll** natychmiast przywraca automatyczne śledzenie i kadrowanie ruchu.

Pomiar kończy się automatycznie po upływie zdefiniowanego czasu eksperymentu (np. 10 sekund), co zapobiega powstawaniu nieskończonych linii na wykresie po zatrzymaniu wózka. Czas ten użytkownik może samodzielnie ustawić w milisekundach w dedykowanym polu tekstowym przed uruchomieniem pomiaru. Wpisana wartość jest automatycznie sprawdzana pod kątem błędów, a samo pole blokuje się po kliknięciu przycisku **Start Measurement**, co zapobiega przypadkowym zmianom w trakcie ruchu. Użytkownik może też przerwać rejestrację w dowolnym momencie, klikając przycisk **Cease Measurement**. Po zakończeniu pracy interfejs wraca do stanu spoczynku, a przyciski narzędziowe są po-

nownie odblokowywane.

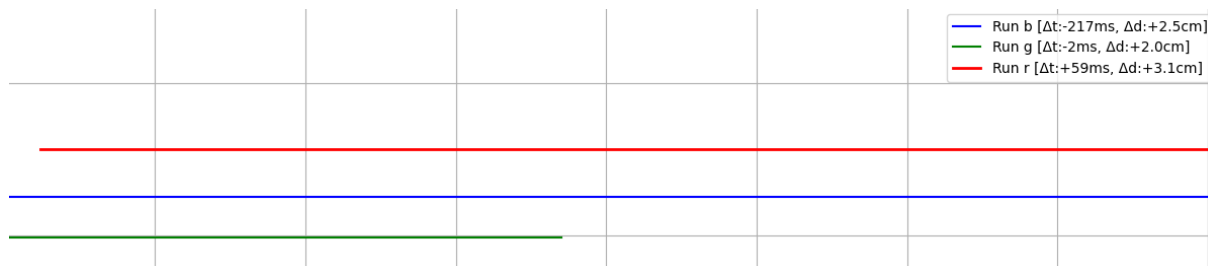
Kluczową funkcjonalnością z punktu widzenia fizyki eksperymentalnej jest możliwość porównywania wielu serii pomiarowych. Każde kolejne kliknięcie **Start Measurement** (np. po zmianie kąta nachylenia równi, właściwości oscylatora lub masy wózka) generuje na tym samym wykresie nową linię o unikalnym, automatycznie dobranym kolorze, która zostaje dopisana do dynamicznej legendy. Aby ułatwić wizualną ocenę szybkości zmian badanych wielkości dynamicznych, użytkownik może zaznaczyć opcję **Align to (0,0)**. Powoduje to matematyczne przesunięcie nowych krzywych na wykresie w taki sposób, aby ich rzeczywiste punkty startowe nałożyły się idealnie w środku układu współrzędnych. Umożliwia to bezpośrednie i precyzyjne porównanie nachylenia parabol lub faz oscylacji różnych serii pomiarów.



Rysunek 5: Nałożenie wielu krzywych na jeden wykres.

Ponieważ automatyczna synchronizacja do punktu zero nakłada na siebie jedynie pierwsze zarejestrowane próbki i nie uwzględnia opóźnień wynikających z ludzkiej reakcji (np. zwolnienia wózka lub wahadła ułamek sekundy po starcie stopera), program oferuje zaawansowaną funkcję manualnej korekty geometrii wykresów za pomocą techniki przeciągnij i upuść. Po zakończeniu zbierania danych użytkownik może kliknąć lewym przyciskiem myszy bezpośrednio na wybraną linię trajektorii. Wybrana seria zostaje optycznie pogrubiona, a trzymanie wciśniętego lewego przycisku pozwala na jej płynne przesuwanie

w płaszczyźnie poziomej (oś czasu) oraz pionowej (oś odległości). W trakcie tego ruchu legenda wykresu dynamicznie przekształca się w aktywny wskaźnik pomiarowy, wyświetlając w czasie rzeczywistym aktualne przesunięcie czasowe w milisekundach oraz przeszerzenie w centymetrach. Pozwala to na idealne, ręczne zsynchronizowanie rzeczywistych momentów rozpoczęcia ruchu dla wszystkich serii. Po dopasowaniu wykresów, kliknięcie lewym przyciskiem myszy w dowolne puste tło układu współrzędnych powoduje bezpieczne odznaczenie linii i przywrócenie jej standardowej grubości.



Rysunek 6: W wyniku przesunięcia krzywej legenda zmienia się (patrz prawy górny róg) uwzględniając przesunięcie w osi czasu oraz osi odległości.

Po zakończeniu serii doświadczeń i pełnej synchronizacji krzywych, użytkownik może skorzystać z przycisku **Save Plot**, który otwiera systemowe okno menedżera plików, pozwalając na zapisanie wygenerowanego wykresu jako pliku graficznego (PNG lub JPG) do późniejszego wykorzystania w sprawozdaniu. Kliknięcie przycisku **Clear Plot** całkowicie czyści obszar roboczy z linii, wskaźników przesunięć i legendy, przywracając domyślną skalę i resetując kolejkę kolorów przed nową serią badawczą, natomiast przycisk **Quit** bezpiecznie zamyka połączenie z czujnikiem i wyłącza aplikację.

2.4 Opis błędów oraz rozwiązania

W trakcie używania systemu pomiarowego mogą pojawić się problemy techniczne wynikające z błędów połączenia sprzętowego lub nieprawidłowego wpisania parametrów w oknie programu. Poniżej znajduje się lista najczęstszych komunikatów awaryjnych, które aplikacja obsługuje, wraz z prostą instrukcją, jak naprawić daną usterkę.

1. Komunikat: "No serial ports detected" lub "Arduino not found" (Wyjątek *SerialException*)

Kiedy występuje: Błąd pojawia się natychmiast po uruchomieniu aplikacji systemowej, jeśli program nie potrafi znaleźć żadnego aktywnego urządzenia na magistrali USB.

Przyczyna: Kabel USB od Arduino jest odłączony od komputera, port USB w komputerze jest uszkodzony, płytkę nie ma zasilania lub w systemie operacyjnym brakuje odpowiednich sterowników (częsty problem w przypadku klonów Arduino z chipem CH340).

Rozwiązanie:

- Odłącz i podłącz ponownie kabel USB do innego portu w komputerze.
- Upewnij się, że na płytce Arduino świeci się dioda zasilania (ON).

- Zainstaluj sterowniki do układu CH340, jeśli komputer w ogóle nie widzi urządzenia.
- Jeśli pracujesz na systemie Linux (np. Ubuntu), upewnij się, że masz uprawnienia do czytania portu szeregowego (komenda `sudo usermod -a -G dialout $USER` i ponowne zalogowanie).

2. **Komunikat: "Wprowadź poprawną, dodatnią liczbę milisekund!" (Wyjątek *ValueError* w polu *Timeout*)**

Kiedy występuje: Błąd pojawia się po kliknięciu przycisku *Start Measurement*, jeśli użytkownik zmodyfikował pole tekstowe czasu pomiaru.

Przyczyna: W pole *Timeout [ms]* wpisano tekst zamiast liczb (np. dziesięć sekund), pole pozostawiono całkowicie puste lub wpisano wartość mniejszą bądź równą zero (np. -5000).

Rozwiązanie: Kliknij przycisk zamknięcia błędu, wyczyść pole tekstowe czasu i wpisz poprawną liczbę całkowitą określającą czas eksperymentu w milisekundach (np. 10000 dla 10 sekund lub 5000 dla 5 sekund). Program automatycznie odrzuci spacje i znaki niedrukowalne.

3. **Wykres zawiesza się na komunikacie: "Waiting for data..." i nie rysuje linii**

Kiedy występuje: Program uruchamia się poprawnie, port szeregowy zostaje otwarty, ale po kliknięciu przycisku startu lub zwolnieniu przycisku na równi wykres nie rusza.

Przyczyna: Brak fizycznej komunikacji między czujnikiem LIDAR a Arduino (błędne podłączenie pinów) lub niepoprawny format danych. Program uruchomił pętlę, ale funkcja filtrująca odrzuca puste ramki lub urwane teksty (np. ,9), czekając na pierwszy poprawny pakiet.

Rozwiązanie:

- Sprawdź, czy zielony i biały przewód lasera nie poluzowały się w pinach 10 i 11.
- Upewnij się, że czujnik nie jest niczym zasłonięty w odległości mniejszej niż 2 cm (strefa martwa).
- Zresetuj całe urządzenie poprzez odłączenie i ponowne podłączenie kabla USB, a następnie wyczyść wykres przyciskiem *Clear Plot* i ponów próbę.

4. **Komunikat w konsoli: "Odrzucono anomalny dystans / Pominięto uszkodzone dane"**

Kiedy występuje: W trakcie trwania pomiaru lub na samym jego starcie, w oknie konsoli tekstowej pojawiają się pojedyncze ostrzeżenia.

Przyczyna: Chwilowe zakłócenia elektromagnetyczne na kablu USB (błędy bitowe w transmisji) lub ucieczka wózka z toru optycznego lasera (odbicie wiązki od ściany laboratorium powyżej X metrów).

Rozwiązanie: Te komunikaty są informacją o prawidłowym działaniu filtrów aplikacji. Program automatycznie wychwytyuje te błędy, wyrzuca je do kosza i nie

pozwała im zepsuć wykresu ani zawiesić systemu. Nie musisz nic robić, o ile wykres rysuje się dalej. Jeśli anomalii jest za dużo, popraw stabilność toru wózka i upewnij się, że kable nie leżą obok źródeł silnych zakłóceń (np. zasilaczy sieciowych).

3 Zaimplementowana logika systemu

3.1 Logika agregacji danych

Logiczne i Fizyczne Podstawy Algorytmu Agregacji

Głównym zadaniem tego algorytmu jest inteligentna redukcja i uśrednianie danych w locie. Aby zrozumieć potrzebę jego zastosowania, musimy spojrzeć na rzeczywistość fizyczną eksperymentu. Czujnik laserowy (LIDAR) mierzy odległość z ogromną częstotliwością. Przesyła np. 200 pomiarów na sekundę. W skali mikro (milisekundowej) ruch wózka na równi pochyłej nie jest idealnie gładki. Mamy styczność z drganiami mechanicznymi, mikroskopijnymi nierównościami powierzchni, a sam czujnik ma swoją cyfrową niedoskładność (szum), przez co odczyty mogą nieznacznie *skakać* w górę i w dół o milimetry. Gdybyśmy wrzucili te surowe dane bezpośrednio na wykres, otrzymalibyśmy grubą, *poszarpaną* linię, z której trudno odczytać dokładny kształt krzywej kinematycznej. Algorytm agregacji działa jak filtr wygładzający, który porządkuje te dane w logiczne pakiety.

```
1 AGGR = 10
2 def aggregate_data(times, distances):
3     if not times:
4         return [], []
5
6     aggregated_times = []
7     aggregated_distances = []
8
9     current_time = times[0]
10    sum_distance = 0
11    count = 0
12
13    for t, d in zip(times, distances):
14        if t - current_time <= AGGR:
15            sum_distance += d
16            count += 1
17        else:
18            aggregated_times.append(current_time + AGGR // 2)
19            aggregated_distances.append(sum_distance / count)
20
21            current_time = t
22            sum_distance = d
23            count = 1
24
25    aggregated_times.append(current_time + AGGR // 2)
26    aggregated_distances.append(sum_distance / count)
27
28    return aggregated_times, aggregated_distances
```

Koncepcja *Koszyków Czasowych* (Time Slotting)

Wyobraź sobie, że oś czasu eksperymentu dzielimy na równe odcinki (okienka) o szerokości zdefiniowanej przez stałą AGGR (np. 10 milisekund). Algorytm traktuje te okienka jak fizyczne koszyki.

Gdy rozpoczyna się pomiar, algorytm otwiera pierwszy koszyk na punkcie startowym (np. $t = 12ms$). Przez następne 10 milisekund (czyli dla wszystkich danych, które napłyną do momentu $t = 22ms$), algorytm nie rysuje niczego na wykresie. Zamiast tego wrzuca wszystkie odczyty odległości do tego jednego koszyka, sumuje je i liczy, ile ich wpadło. W ten sposób zbiera grupę reprezentantów dla małego wycinka czasu.

Logika Podejmowania Decyzji (Granica Koszyka)

Algorytm analizuje każdą nadchodzącą paczkę danych i zadaje sobie pytanie retoryczne: *Czy ten punkt nadal mieści się w moim obecnym koszyku?*

1. Odpowiedź: TAK $\rightarrow$ Punkt jest akumulowany. Zwiększamy łączną sumę odległości i licznik próbek.
2. Odpowiedź: NIE $\rightarrow$ Nadchodzi moment zwrotny. Ten punkt (który właśnie przekroczył granicę, np. ma czas $23ms$) staje się sygnałem do zamknięcia starego koszyka.

W tym momencie algorytm wykonuje kluczową operację matematyczno-fizyczną: zamienia całą grupę punktów z koszyka w jeden, idealny, uśredniony punkt reprezentacyjny.

Dlaczego Środek Okna? (Logika współrzędnej czasu X)

Gdy koszyk jest zamykany, algorytm musi zdecydować, w którym miejscu na osi czasu (X) postawić punkt reprezentacyjny.

- Mógłby wybrać czas pierwszego punktu z koszyka.
- Mógłby wybrać czas ostatniego punktu z koszyka.
- Wybiera środek geometryczny okna ($current_time + AGGR//2$).

Dlaczego to logiczne z punktu widzenia fizyki? Ponieważ czujnik wysyła dane asynchronicznie. W jednym oknie 10-milisekundowym punkty mogą zgrupować się na początku, a w innym na końcu. Przypisanie uśrednionej odległości do idealnego środka przedziału czasowego gwarantuje, że punkty na wykresie będą rozłożone w idealnie równych odstępach czasu (np. co 10 ms). Zapobiega to zniekształceniom kształtu paraboli (ruchu przyspieszonego), które mogłyby sfalszować obliczenia prędkości.

Tłumienie Szumu (Logika współrzędnej odległości Y)

Współrzędna odległości (Y) punktu reprezentacyjnego to średnia arytmetyczna wszystkich odczytów z koszyka ($\bar{d} = \frac{\sum d}{n}$).

- Fizyczny sens: Jeśli w ciągu tych 10 milisekund czujnik podał wartości: $50.1cm$, $49.9cm$, $50.0cm$, $50.2cm$ oraz jeden zaszumiony skok $51.0cm$, to wyliczona średnia wyniesie około $50.24cm$.
- Wpływ chwilowego, pojedynczego błędu (szumu) został drastycznie osłabiony przez pozostałe, stabilne pomiary. Wykres staje się gładką, ciągłą linią o wysokiej jakości estetycznej i naukowej.

Logika *Domykania* (Ostatni Koszyk)

Na samym końcu, gdy eksperyment dobiega końca lub użytkownik kliknie ***Cease measurement*** pętla analizująca dane kończy pracę. W tym momencie w ostatnim koszyku mogły zostać np. 3 punkty, które nie doczekały się oficjalnego zamknięcia (bo nie nadszedł już żaden kolejny punkt, który wybiłby algorytm do sekcji **else**). Logika algorytmu przewiduje ten scenariusz: instrukcje umieszczone tuż przed wyjściem z funkcji wymuszają awaryjne uśrednienie i opróżnienie tego, co zostało w ostatnim koszyku. Dzięki temu żaden, nawet ostatni milimetr ruchu wózka na końcu równi pochyłej, nie zostanie zgubiony.

Podsumowanie korzyści logicznych dla fizyka:

Algorytm ten dokonuje kompresji danych z zachowaniem praw fizyki. Zamiast obciążać komputer rysowaniem tysięcy surowych, drgających pikseli, przekształca strumień laboratoryjny w idealną matematycznie serię punktów, idealnie przygotowaną pod wyznaczenie wektora prędkości czy przyspieszenia ziemskiego.

3.2 Logika filtrowania i odrzucania wartości anomalnych

W trakcie przesyłania danych przez port szeregowy oraz podczas pracy lasera w warunkach laboratoryjnych mogą pojawić się zakłócenia. Objawiają się one nagłym odebraniem przez program absurdalnych wartości, na przykład czasu ujemnego lub odległości rzędu kilkunastu metrów (mimo że tor ma zaledwie dwa metry). Zakłócenia te wynikają najczęściej z przekłamań pojedynczych bitów w kablu USB (tzw. błędy transmisji) lub z chwilowej utraty celu przez laser (np. gdy wiązka odbije się od ściany lub sufitu, bo wózek lekko drgnął na boki).

Aby te fałszywe punkty (anomalie) nie zepsuły automatycznego skalowania wykresu i nie zniekształciły wyników, program stosuje dwa proste filtry bezpieczeństwa w domenie czasu i przestrzeni. Filtrowanie to odbywa się wewnątrz metody `_read_serial_line`:

1. **Filtr anomalii czasu:** Jeśli program obliczy czas relatywny t i wyjdzie on mniejszy od zera ($t < 0$) lub dwukrotnie większy niż ustawiony czas pomiaru ($t > 2 \cdot T_{timeout}$), punkt jest ignorowany. Pierwszy warunek chroni przed błędami cofania się czasu, a drugi odcina szum transmisyjny. Zmienna ta jest kontrolowana w kodzie przez parametr `self.current_timeout`.
2. **Filtr fizycznego zasięgu lasera:** Program sprawdza, czy odczytany dystans mieści się w sztywnych granicach [`MIN_DISTANCE`, `MAX_DISTANCE`], zdefiniowanych jako 2 cm oraz 500 cm. Wartość poniżej 2 cm oznacza wejście w strefę martwą czujnika, a powyżej 500 cm to ewidentne odebranie odbicia od elementów tła laboratorium.

Jeśli jakakolwiek próbka nie spełni tych kryteriów, funkcja odczytu zwraca wartość `None`, co dla pętli głównej jest sygnałem do pominięcia tego punktu i przejścia do kolejnej iteracji.

Poniżej przedstawiono fragment kodu z metody `_read_serial_line`, który bezpośrednio odpowiada za tę logikę:

```

1 try:
2     t_raw = int(parts[0].strip())
3     d = int(parts[1].strip())
4     btn_state = int(parts[2].strip())
5 except ValueError:
6     print(f"Pominieto uszkodzone dane transmisyjne: {data}")
7     return None
8
9 # Jedyne filtry na tym etapie: fizyczny zakres pracy sensora LIDAR
10 MIN_DISTANCE = 2      # cm
11 MAX_DISTANCE = 300    # cm
12 if d < MIN_DISTANCE or d > MAX_DISTANCE:
13     return None
14
15 return t_raw, d, btn_state

```

Uzupełnieniem tego mechanizmu jest centralne sprawdzenie czasu w głównej pętli pomiarowej metody `update_plot`, zaprezentowane na poniższym listingu:

```

1 # Obliczenie czasu relatywnego
2 t = t_raw - t0
3
4 # Filtr anomalii chroniący przed rzadkimi błędami skoków rejestru
5 if t < 0 or t > self.current_timeout * 2.0:
6     continue

```

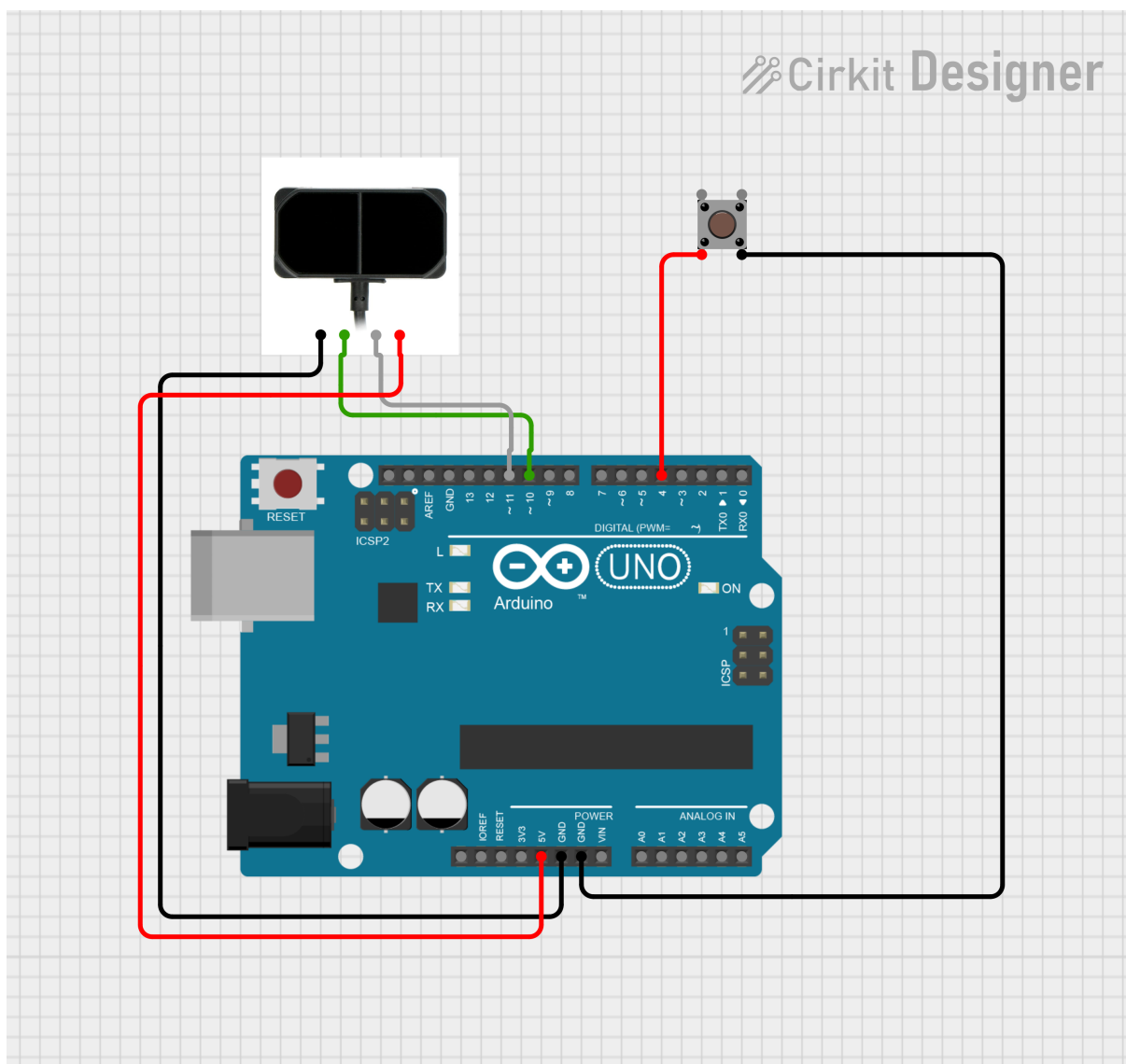
Dzięki takiemu podziałowi ról, surowy strumień danych jest skutecznie oczyszczany przed jakimkolwiek rysowaniem. Wykres pozostaje idealnie wykadrowany na rzeczywisty ruch wózka lub oscylatora, a baza danych w tablicach `raw_ts` oraz `raw_ds` jest wolna od zakłóceń, co gwarantuje pełną poprawność i stabilność działania programu przy każdym kolejnym powtórzeniu eksperymentu.

UWAGA: Wstępne ograniczanie odległości odbywa się już na poziomie płytki Arduino w pliku `main.cpp`. Zastosowany tam warunek *if(distance < 600)* odrzuca wszystkie odczyty powyżej 600 centymetrów (6 metrów). Jest to podyktowane fizyczną specyfikacją czujnika LIDAR, którego maksymalny zasięg stabilnej pracy wewnątrz pomieszczeń wynosi około 12 metrów, natomiast w warunkach laboratoryjnych wartości powyżej 6 metrów są traktowane jako błąd i bezużyteczny szum pomiarowy, którego nie warto przysyłać do komputera.

4 Opis sprzętu oraz połączeń

Komponent	Opis
Arduino Uno	Główny mikrokontroler sterujący pracą systemu.
Benewake TFmini Plus	Laserowy czujnik odległości klasy LiDAR działający w technologii ToF (Time-of-Flight).
Przycisk	Opcjonalna funkcja rozpoczęcia pomiaru.
Przewody połączeniowe	Elastyczne przewody sygnałowe i zasilające (raster 1,27 mm od strony czujnika).

Tabela 1: Zestawienie głównych komponentów systemu.



Rysunek 7: Schemat połączeń układu

Pin	Podłączony element	Opis
10 (RX)	TFmini Plus - Pin TX (Zielony)	Cyfrowa linia odbioru danych (UART).
11 (TX)	TFmini Plus - Pin RX (Biały)	Cyfrowa linia nadawania danych (UART).
GND	TFmini Plus - Pin GND	Wspólna masa (uziemiaenie) układu.
5V	TFmini Plus - Pin VCC	Szyna zasilania sensora laserowego.
4	Przycisk (Styk A)	Sygnał wejściowy (funkcja INPUT_PULLUP).
GND	Przycisk (Styk B)	Zwarcie pinu do masy po wciśnięciu.

Tabela 2: Opis połączeń pinów dla modułu TFmini Plus i przycisku.

Zasilanie układu

Układ pomiarowy zasilany jest napięciem stałym o wartości $5V (\pm 0,5V)$ dostarczany bezpośrednio z dedykowanego pinu mikrokontrolera Arduino Uno. Maksymalny pobór prądu przez czujnik laserowy TFmini Plus wynosi $110mA$ przy stałej pracy pomiarowej. Stabilizacja napięcia zasilania jest kluczowa dla zachowania deklarowanej przez producenta dokładności oraz uniknięcia błędów nasycenia fotodiody przy silnym świetle odbitym.

Parametry techniczne

Szczegółowe specyfikacje czujnika odległości, wyznaczone na podstawie rzeczywistych ograniczeń laboratoryjnych oraz danych producenta, zestawiono w poniższej tabeli. Kompletna specyfikacja i dokumentacja techniczna urządzenia zawarta jest w osobnym pliku: `TFminiPlus_datasheet.pdf`.

Parametr	Wartość fabryczna / Zakres
Zakres roboczy (zasięg)	Od $0,1m$ do $12m$
Dokładność pomiarowa	$\pm 5cm$ (do $6m$), 1% (powyżej $6m$)
Częstotliwość pracy (odświeżanie)	Od $1Hz$ do $1000Hz$ (regulowana)
Kąt widzenia (FOV)	$3,6^\circ$
Napięcie zasilania	$5V \pm 0,5V$
Interfejsy komunikacyjne	UART, I^2C
Klasa szczelności obudowy	IP65
Masa modułu	$11g$

Tabela 3: Specyfikacja techniczna i parametry robocze czujnika TFmini Plus.

5 Alternatywne Zastosowania Systemu Pomiarowego

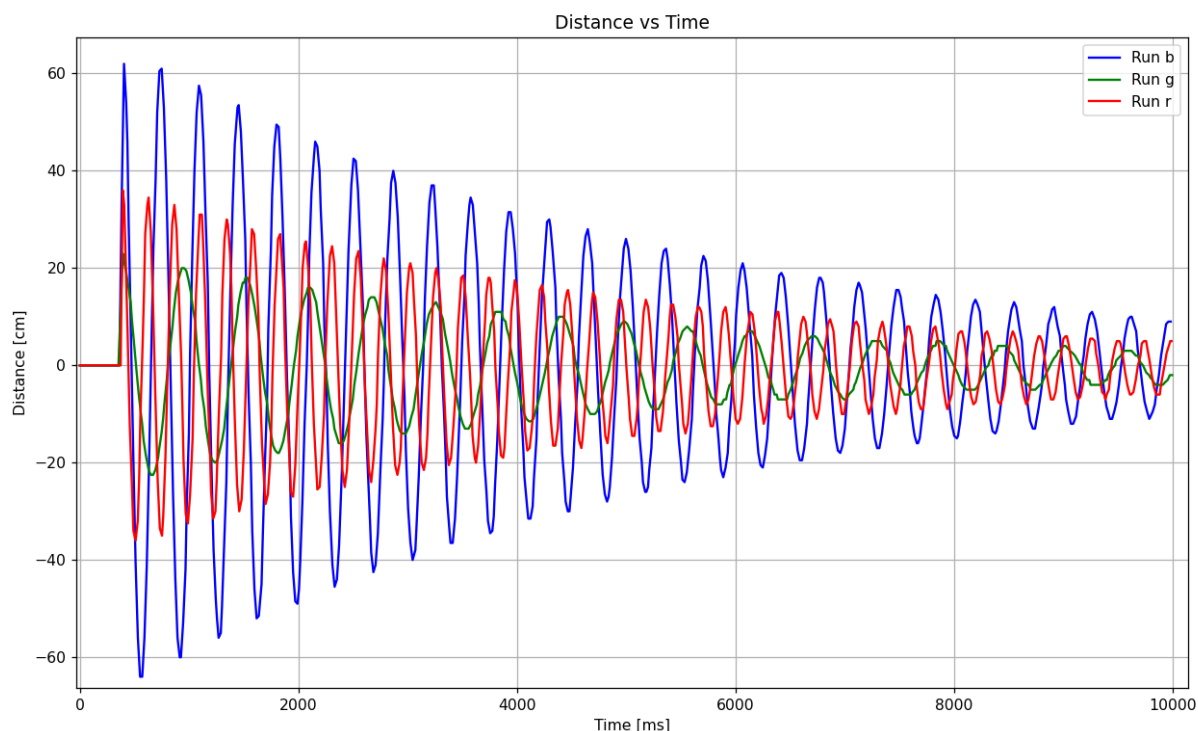
5.1 Badanie i Porównywanie Poziomych (Pionowych) Oscylatorów Harmonicznych

Dzięki uniwersalnej architekturze filtracji danych oraz zaawansowanym algorytmom manipulacji geometrią wykresu w czasie rzeczywistym, oprogramowanie to wykracza poza ramy badania ruchu postępowego na równi pochyłej i może być z powodzeniem stosowane w innych kluczowych działach fizyki klasycznej. Wyjątkowo wartościowym dydaktycznie i naukowo zastosowaniem systemu jest analiza porównawcza poziomych oscylatorów harmoniczych (ciężarków na sprężynach poruszających się po torze o minimalnym tarcu). W tego typu doświadczeniach kluczowym wyzwaniem metrologicznym jest precyzyjne określenie położenia równowagi jako fizycznego punktu zero oraz zsynchronizowanie fazy ruchu drgającego dla układów o różnych właściwościach fizycznych. Poniższa instrukcja krok po kroku opisuje metodologię przeprowadzenia takiego eksperymentu za pomocą przygotowanego narzędzia.

W pierwszej kolejności użytkownik musi przygotować stanowisko badawcze i uruchomić aplikację. Przed rozpoczęciem jakiegokolwiek rejestracji danych, kluczowe jest aktywowanie na panelu sterowania funkcji **Align to (0,0)** oraz fizyczne umieszczenie ciężarka w stanie spoczynku, dokładnie w jego naturalnym położeniu równowagi. W tej pozycji siła sprężystości sprężyny oraz siła ciężkości równoważą się, a odległość mierzona przez czujnik laserowy stanowi bazę odniesienia dla całego późniejszego ruchu.

Procedurę pomiarową rozpoczyna się poprzez kliknięcie przycisku **Start Measurement**, gdy ciało wciąż znajduje się w bezruchu w położeniu równowagi. Jest to krok o charakterze krytycznym z punktu widzenia logiki programu. Ponieważ funkcja wyrównywania pobiera absolutnie pierwszy odebrany punkt pomiarowy jako środek nowego układu współrzędnych, program natychmiast przypisuje aktualną pozycję spoczynkową jako matematyczny punkt zero w przestrzeni i czasie. Dopiero po ułamku sekundy od kliknięcia przycisku i zarejestrowaniu punktu startowego, użytkownik manualnie wychyla ciężarek z położenia równowagi o pożądaną amplitudę, po czym swobodnie puszcza oscylator.

W tym momencie na ekranie komputera zaczyna rysować się klasyczna sinusoida ruchu drgającego, odzwierciedlająca zmiany położenia ciała w funkcji czasu. Ponieważ czas i droga zaczęły być naliczane od punktu równowagi, wykres idealnie symetrycznie obrazuje wychylenia w kierunku dodatnim (od czujnika) oraz ujemnym (w stronę czujnika), co pozwala na bezpośredni odczyt amplitudy drgań bez konieczności wykonywania ręcznych obliczeń różnicowych. Po zarejestrowaniu kilku pełnych okresów drgań, użytkownik przerywa proces za pomocą przycisku **Cease Measurement** lub pozwala programowi na automatyczne zakończenie pomiaru po upływie zdefiniowanego czasu eksperymentu.



Rysunek 8: Zasymulowany ruch harmoniczny (`serial_dev_harmonics.py`)

W celu dokonania analizy porównawczej, użytkownik – pod żadnym pozorem nie klikając przycisku ***Clear Plot*** – zmienia właściwości fizyczne oscylatora, na przykład poprzez podpięcie sprężyny o innej stałej sztywności k lub zmianę masy drgającego ciężarka. Następnie powtarza dokładnie całą procedurę: ustawia nowe ciało w jego własnym położeniu równowagi, uruchamia pomiar, wychyla je o taką samą lub inną amplitudę i puszcza wolno.

Na jednym ekranie komputera pojawiają się wówczas dwa lub więcej nałożone na siebie wykresy o różnych, automatycznie dobranych kolorach. Dzięki temu, że oba wykresy zostały automatycznie wyzerowane w swoich unikalnych położeniach równowagi, użytkownik uzyskuje bezprecedensową możliwość bezpośredniego porównania parametrów dynamicznych obu układów. Może natychmiastowo ocenić wzrokowo różnicę w okresie i częstotliwości drgań dla sprężyn o różnej sztywności oraz przeanalizować stopień tłumienia drgań w czasie. Ewentualne przesunięcia fazowe wynikające z niesynchronicznego puszczenia ciężarków przez człowieka można w ułamku sekundy skorygować manualnie za pomocą funkcji chwytania linii lewym przyciskiem myszy i precyzyjnego przeciągania jej wzdłuż osi czasu, co czyni to oprogramowanie niezwykle potężnym i uniwersalnym narzędziem w rękach fizyka eksperymentatora.

6 Podsumowanie

Najważniejsze cechy projektu

Najważniejsze cechy projektu to jego uniwersalność, bezbłędna stabilność działania oraz prostota obsługi. System łączy precyzję pomiaru laserowego z zaawansowanym filtrowaniem danych na żywo. Kluczowymi zaletami oprogramowania są automatyczna kompresja i wygładzanie zaszumionego sygnału, pełne zabezpieczenie przed awariami transmisji szeregowej, a także unikalna funkcja ręcznego przesunięcia i nakładania gotowych linii na wykresie za pomocą myszki.

Osiągnięte rezultaty

Osiągnięte rezultaty to stworzenie w pełni sprawnego, bezbłędnie działającego stanowiska laboratoryjnego. Udało się całkowicie wyeliminować problem zawieszania programu przy starcie oraz błędy ujemnego czasu generowane przez zapchane bufory sprzętowe Arduino i błędy ludzkiego refleksu. Urządzenie z pełną dokładnością rejestruje trajektorię ruchu ciał w czasie rzeczywistym i pozwala na bezpośrednie, wzrokowe porównywanie parametrów dynamicznych różnych doświadczeń, co potwierdziły testy z wózkiem na równi oraz oscylatorem harmonicznym.

Możliwe kierunki rozwoju

Możliwe kierunki rozwoju projektu obejmują przede wszystkim dodanie funkcji automatycznego dopasowywania krzywej (regresji kwadratowej) do wyznaczania przyspieszenia oraz numerycznego różniczkowania drogi w celu rysowania wykresów prędkości i przyspieszenia w dodatkowych zakładkach. Warto również rozważyć dodanie modułu automatycznego zapisu wyników do plików CSV oraz implementację bezprzewodowej transmisji danych przez moduł Bluetooth, co całkowicie uwolniłoby stanowisko pomiarowe od kabli USB.

Repozytorium GitHub

Wszelkie kody źródłowe zostały zarchiwizowane i udostępnione publicznie na licencji MIT.

Adres URL: <https://github.com/Z1moRG/InclinedPlane>